

App Development Project Report

1. Student Details

Name: Rishab Arora

Roll Number: 25F2008178

Email: 25f2008178@ds.study.iitm.ac.in

About Me: I'm Rishab Arora, a student doing the BS in Data Science & Applications program at IIT Madras and the BTech CSE Program at Maharaja Surajmal Institute of Technology (GGSIPU), New Delhi.

2. Project Details

Project Title: Placement Portal Application

Problem Statement & Approach:

Educational institutions require a centralized, automated infrastructure to orchestrate the complex ecosystem of campus recruitment. Relying on fragmented manual processes - such as spreadsheets and email threads - creates data silos that lead to administrative bottlenecks, registration errors, and a lack of real-time visibility into placement metrics.

The app was built using Flask as the backend framework with a modular structure. It allows the admin to view all ongoing activities and approve the creation of placement drives, while students can easily view all the drives, upload their resumes, and await the status updates.

3. AI/LLM Declaration

No use of AI/LLM.

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core backend web framework
SQLAlchemy	Object Relational Mapper for SQLite database
Jinja2	Template engine for rendering dynamic HTML pages
Bootstrap 5	Frontend styling and responsive design
Chart.js	To visualize the application statuses for the college admin

Flask-Security	User authentication and session management
WTForms	Frontend form validation
SQLite	Lightweight local database for storing user data

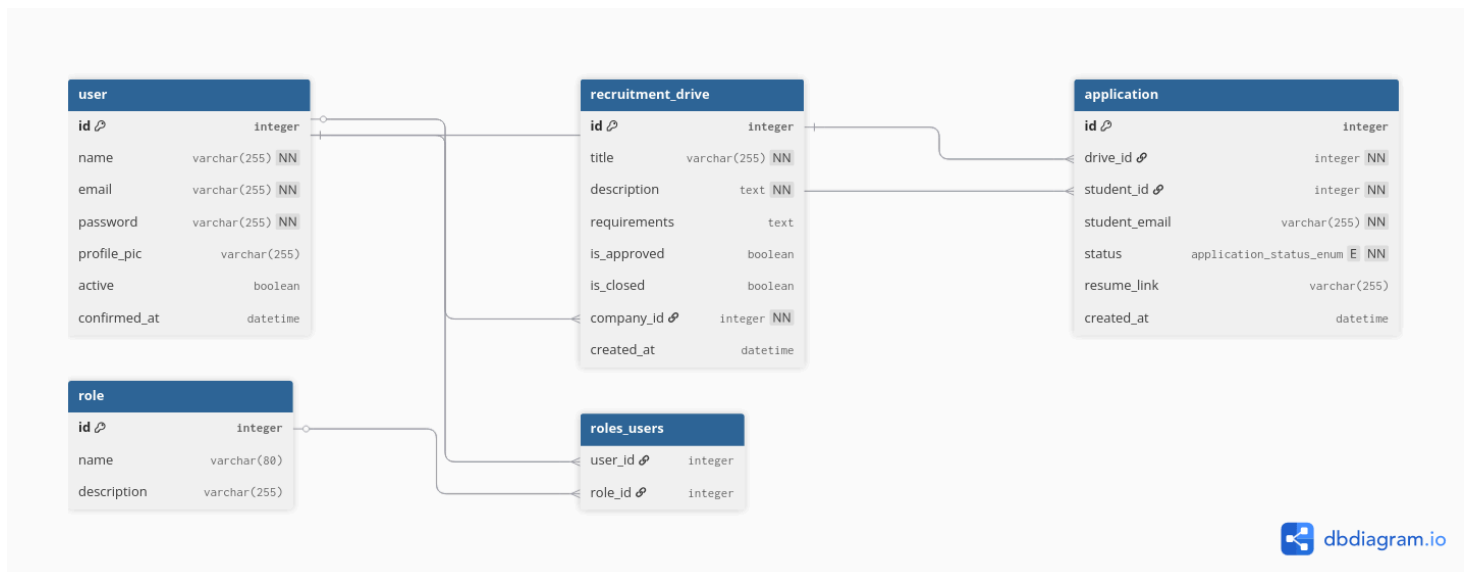
5. Database Schema / ER Diagram

Tables:

1. **User:** stores user credentials and profile details (`id`, `name`, `email`, `password`, `profile_pic`).
2. **Role:** defines access permissions for different user types (`id`, `name`, `description`).
3. **RecruitmentDrive:** stores company-posted job opportunities (`id`, `title`, `description`, `requirements`, `is_approved`, `is_closed`, `company_id`, `created_at`).
4. **Application:** logs student submissions for specific drives (`id`, `drive_id`, `student_id`, `status`, `resume_link`, `created_at`).

Relationships:

- **Many-to-Many** → **User** ↔ **Role** (via `roles_users` join table)
- **One-to-Many** → **User (Company)** → **RecruitmentDrive**
- **One-to-Many** → **User (Student)** → **Application**
- **One-to-Many** → **RecruitmentDrive** → **Application**



6. API Resource Endpoints

Endpoint	Method	Description
Placement & Drives		
<code>/api/placement/create_drive</code>	POST	Allows companies to submit a new recruitment drive for approval.
<code>/api/placement/get_drives</code>	GET	Fetches all available recruitment drives.
<code>/api/placement/get_drives/<int:drive_id></code>	GET	Retrieves detailed information for a specific drive.
<code>/api/placement/my_drives</code>	GET	Fetches drives created by the currently logged-in company.
<code>/api/placement/approve_drive/<id></code>	PATCH	Admin endpoint to approve a pending recruitment drive.
<code>/api/placement/reject_drive/<id></code>	PATCH	Admin endpoint to reject a recruitment drive.
<code>/api/placement/edit_drive/<id></code>	PUT	Update details of an existing recruitment drive.
<code>/api/placement/remove_drive/<id></code>	DELETE	Permanently remove a recruitment drive from the portal.
Applications		
<code>/api/application/apply/<int:drive_id></code>	POST	Enables a student to apply for a specific recruitment drive.
<code>/api/application/my-applications</code>	GET	Fetches the application history for the logged-in student.
<code>/api/application/update-status/<int:application_id></code>	PATCH	Updates the hiring status (e.g., Shortlisted, Selected).
Student Management		
<code>/api/student/get_students</code>	GET	Fetches a list of all registered students.
<code>/api/student/search_students/by_id/<id></code>	GET	Find a specific student using their unique ID.

<code>/api/student/search_students/by_name/<name></code>	GET	Search for students by their full or partial name.
<code>/api/student/search_students/by_email/<email></code>	GET	Search for a student using their email address.
<code>/api/student/activate/<id></code>	PATCH	Admin endpoint to enable a student's account.
<code>/api/student/deactivate/<id></code>	PATCH	Admin endpoint to disable a student's account.

Company Management

<code>/api/company/get_companies</code>	GET	Fetches a list of all registered companies.
<code>/api/company/search_companies/by_id/<id></code>	GET	Find a specific company using their unique ID.
<code>/api/company/search_companies/by_name/<name></code>	GET	Search for companies by their corporate name.
<code>/api/company/search_companies/by_email/<email></code>	GET	Search for a company using their corporate email.
<code>/api/company/activate/<id></code>	PATCH	Admin endpoint to enable a company's account.
<code>/api/company/deactivate/<id></code>	PATCH	Admin endpoint to disable a company's account.

YAML API Definition File:

Included separately in the submission ZIP as `api.yaml`.

7. Video Presentation

Drive Link:

https://drive.google.com/file/d/1VTiYSY02asogfnXwfbI9g8yltX9h4yjP/view?usp=drive_link